

# Emotion-Aware Voice Agent

## Modular Project Structure

### Directory Layout

emotion-voice-agent/

■

■■■■ README.md  
■■■■ requirements.txt  
■■■■ pyproject.toml  
■■■■ .env.example  
■■■■ .gitignore

template for API keys (never commit .env)

■

■■■■ configs/

■ ■■■■ features.yaml  
■ ■■■■ model.yaml  
■ ■■■■ mood.yaml  
■ ■■■■ memory.yaml  
■ ■■■■ tts.yaml  
■ ■■■■ agent.yaml

MFCC bins, window size, hop length  
hidden size, dropout, LR, epochs  
decay rate, alpha, thresholds, persistence N  
top\_k, importance weight, embedding model  
prosody profiles per mood mode  
orchestrator settings, fallback flags

■

■■■■ data/

■ ■■■■ manifests/  
■ ■■■■ splits/  
■ ■■■■ raw/

CSV: utt\_id, speaker\_id, wav\_path, label, split, duration  
saved train/val/test indices (reproducible)  
.gitignore – RAVDESS/CREMA-D wavs go here locally

■

■■■■ src/

■ ■■■■ \_\_init\_\_.py

■ ■

■ ■■■■ perception/

HEARING + EMOTION DETECTION

■ ■ ■■■■ \_\_init\_\_.py

■ ■ ■■■■ asr.py

Whisper wrapper – transcribe(audio) → text

■ ■ ■■■■ preprocess.py

load, trim, denoise, normalize audio

■ ■ ■■■■ features.py

MFCC/mel extraction + caching + openSMILE

■ ■ ■■■■ datasets.py

PyTorch Dataset/DataLoader, padding, masking

■ ■ ■■■■ emotion\_classifier.py

predict(audio) → prob\_vector

■ ■

■ ■■■■ models/

TRAINABLE MODELS (separate from inference)

■ ■ ■■■■ \_\_init\_\_.py

■ ■ ■■■■ baseline.py

RF / XGBoost on prosodic vector

■ ■ ■■■■ cnn.py

1D CNN (middle baseline)

■ ■ ■■■■ lstm.py

small Bi-LSTM + attention (primary)

■ ■ ■■■■ train.py

training loop, early stopping, checkpointing

■ ■

■ ■■■■ mood/

SHORT-TERM EMOTIONAL STATE

■ ■ ■■■■ \_\_init\_\_.py

■ ■ ■■■■ mood\_manager.py

EMA decay, persistence, threshold logic

■ ■ ■■■■ profiles.py

behavior profiles (EMPATHETIC, CALMING, etc.)

■ ■ ■■■■ mood\_history.py

logs mood vectors per turn for trajectory plots

■ ■

■ ■■■■ memory/

LONG-TERM FACTUAL MEMORY

■ ■ ■■■■ \_\_init\_\_.py

■ ■ ■■■■ memory\_manager.py

high-level API: store, retrieve, reset

■ ■ ■■■■ json\_store.py

JSON backend (Phase 6a)

■ ■ ■■■■ rag\_store.py

Chroma/FAISS + MiniLM backend (Phase 6b)

■ ■ ■■■■ fact\_extractor.py

LLM micro-call: durable fact or NONE?

■ ■ ■■■■ embeddings.py

sentence-transformers embed() wrapper

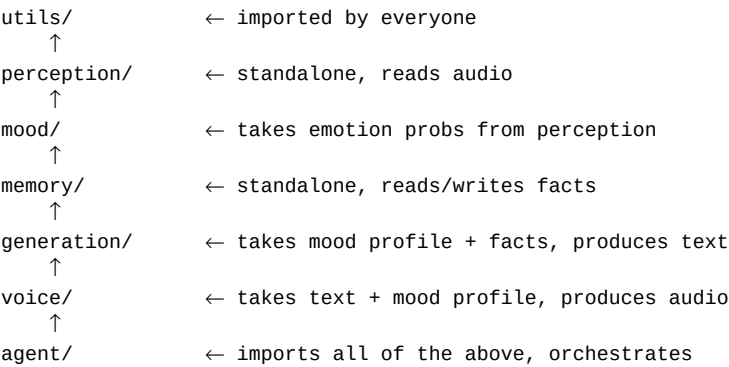
|                                 |                                                  |
|---------------------------------|--------------------------------------------------|
| ■ ■                             |                                                  |
| ■ ■■■ generation/               | RESPONSE GENERATION                              |
| ■ ■ ■■■ __init__.py             |                                                  |
| ■ ■ ■■■ prompt_builder.py       | build_system_prompt(profile, facts) – THE BRAIN  |
| ■ ■ ■■■ llm_client.py           | Gemini API wrapper – _call_llm()                 |
| ■ ■ ■■■ template_generator.py   | deterministic template fallback                  |
| ■ ■ ■■■ response_generator.py   | top-level: USE_LLM toggle, try/except fallback   |
| ■ ■                             |                                                  |
| ■ ■■■ voice/                    | TTS OUTPUT                                       |
| ■ ■ ■■■ __init__.py             |                                                  |
| ■ ■ ■■■ tts_client.py           | sends text + prosody to Colab, receives audio    |
| ■ ■ ■■■ tag_injector.py         | inserts emotion tags based on mood               |
| ■ ■ ■■■ audio_player.py         | plays received audio bytes locally               |
| ■ ■ ■■■ fallback_tts.py         | local pyttsx3 fallback if Colab is down          |
| ■ ■                             |                                                  |
| ■ ■■■ agent/                    | ORCHESTRATOR – TIES EVERYTHING TOGETHER          |
| ■ ■ ■■■ __init__.py             |                                                  |
| ■ ■ ■■■ orchestrator.py         | run_turn() – the full deterministic flow         |
| ■ ■ ■■■ session.py              | ConversationState per user                       |
| ■ ■ ■■■ user_manager.py         | user selection at startup, user_id isolation     |
| ■ ■ ■■■ logger.py               | logs each turn: transcript, emotion, mood, reply |
| ■ ■                             |                                                  |
| ■ ■■■ utils/                    | SHARED UTILITIES                                 |
| ■ ■■■ __init__.py               |                                                  |
| ■ ■■■ audio_utils.py            | mic capture (PyAudio), VAD                       |
| ■ ■■■ config_loader.py          | loads YAML configs, merges defaults              |
| ■ ■■■ timing.py                 | latency measurement per step                     |
| ■ ■                             |                                                  |
| ■ ■■■ notebooks/                |                                                  |
| ■ ■■■ 01_edu.ipynb              | dataset exploration, class distribution          |
| ■ ■■■ 02_train_eval.ipynb       | training + evaluation of all models              |
| ■ ■■■ 03_colab_tts_server.ipynb | CSM-1B inference server on Colab                 |
| ■ ■■■ 04_tts_finetune.ipynb     | CSM-1B LoRA fine-tuning on Elise dataset         |
| ■ ■                             |                                                  |
| ■ ■■■ tests/                    |                                                  |
| ■ ■■■ test_mood_manager.py      |                                                  |
| ■ ■■■ test_memory_manager.py    |                                                  |
| ■ ■■■ test_fact_extractor.py    |                                                  |
| ■ ■■■ test_prompt_builder.py    |                                                  |
| ■ ■■■ test_orchestrator.py      |                                                  |
| ■ ■■■ test_tag_injector.py      |                                                  |
| ■ ■                             |                                                  |
| ■ ■■■ scripts/                  |                                                  |
| ■ ■■■ prepare_data.sh           | download RAVDESS, preprocess, generate manifest  |
| ■ ■■■ train_models.sh           | train all models sequentially                    |
| ■ ■■■ run_agent.py              | entry point                                      |
| ■ ■                             |                                                  |
| ■ ■■■ results/                  |                                                  |
| ■ ■■■ metrics/                  | per-class P/R/F1, confusion matrices             |
| ■ ■■■ figures/                  | mood trajectory plots, model comparisons         |
| ■ ■■■ logs/                     | per-turn interaction logs                        |
| ■ ■                             |                                                  |
| ■ ■■■ checkpoints/              | .gitignore – saved model weights                 |
| ■ ■■■ memories.json             | .gitignore – JSON store                          |
| ■ ■■■ vector_store/             | .gitignore – Chroma/FAISS index                  |

## Module Responsibilities

| Folder | Responsibility | Depends On | Swappable? |
|--------|----------------|------------|------------|
|--------|----------------|------------|------------|

|             |                                          |                               |                                                       |
|-------------|------------------------------------------|-------------------------------|-------------------------------------------------------|
| perception/ | Hear audio → text + emotion              | Nothing                       | Whisper → faster-whisper, Bi-LSTM → Wav2Vec           |
| models/     | Training only — never touched at runtime | perception/ for features      | Add new architectures freely                          |
| mood/       | Short-term emotional state tracking      | perception/ for emotion probs | Change decay/threshold without touching anything else |
| memory/     | Long-term fact storage + retrieval       | Nothing                       | JSON → RAG → cloud DB by changing one backend         |
| generation/ | Build prompt + generate reply text       | mood/ + memory/ for inputs    | Gemini → GPT → Llama by changing llm_client.py        |
| voice/      | Text → spoken audio                      | mood/ for tag injection       | CSM-1B → EmotiVoice → local TTS                       |
| agent/      | Orchestrator — calls everything in order | All of the above              | Thin glue layer, just coordinates                     |
| utils/      | Shared helpers                           | Nothing                       | Used by everyone                                      |

### Import Flow (No Circular Dependencies)



No module imports upward — perception/ never imports from agent/, memory/ never imports from voice/. Any module can be tested in isolation.

### Design Principle

Each folder is one concern. Config-driven behavior (YAML) means tuning decay rates, switching backends (JSON → RAG), or toggling LLM vs template requires zero code changes — just edit the config file.